KLAi

pandas III: group, join and reshape

How do we answer a question that needs several groups and several tables, and trust the total?

AI and Data Science Professional | Session 9 of 24 | Module B: Python for data
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Summarise records by group with groupby and named aggregation, and lay the result out with pivot_table

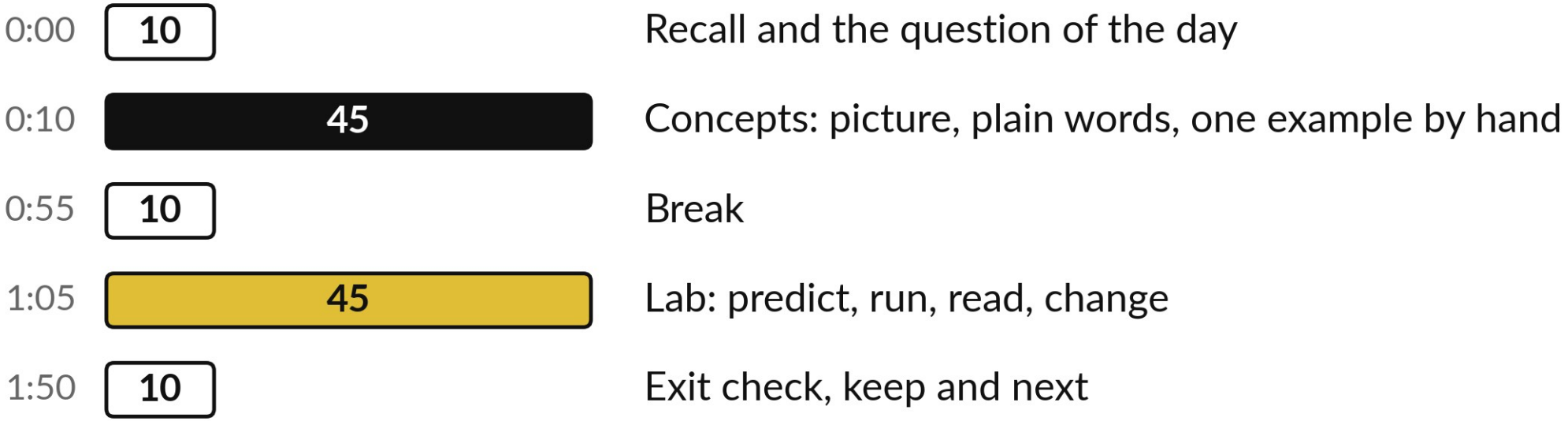


Merge tables on a join key, say what each join type keeps, and prove with row counts and validate that nothing was lost or multiplied



Read months and weekdays from a datetime variable and report revenue by branch, region and year

Your two hours: concepts first, then a Colab lab



Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. The order dates arrive as text. What do you do before you use them as dates?

2. You removed exact duplicate lines from a raw file. What goes in the data cleaning log?

3. Two totals that should agree are different. What do you call that gap?



Answer aloud first. Then we check together.

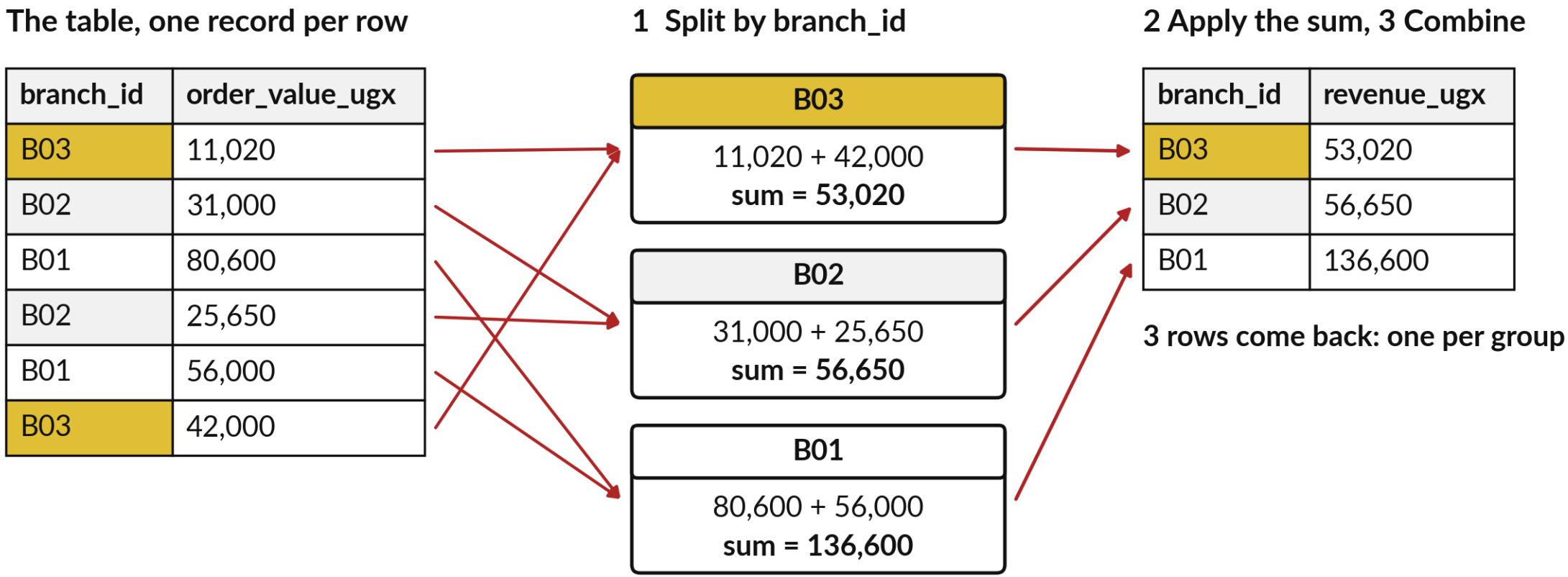
Answers to the three questions from Session 8

Question	Short answer
The order dates arrive as text. What do you do before you use them as dates?	Convert them with pd.to_datetime, then check the data type again to prove it worked.
You removed exact duplicate lines from a raw file. What goes in the data cleaning log?	The rule you used and the row count before and after, so someone else can trust the result.
Two totals that should agree are different. What do you call that gap?	A reconciliation difference. Explain it or reduce it to zero. Never call it a residual, which is a model word.

Words we use today

	Word	What it means
	**groupby and aggregation**	Split records into groups by a key, then reduce each group to one number, such as a sum or a mean.
	**Merge (join) and join key**	Combine two tables by matching a shared variable. The join type says which unmatched records survive.
	**Cardinality (one-to-many, many-to-one)**	How many records on each side match one key value. It decides if a merge keeps, loses or multiplies rows.
	**Fan-out (row multiplication) and lost rows**	Fan-out: a merge repeats a record, so totals inflate. Lost rows: records with no match that an inner join drops.
	**pivot_table (wide and long format)**	A summary grid with one variable down the side and one across the top, which is wide format.

groupby splits the records into groups and reduces each group to one number



An excerpt of six orders. Colours mark the groups. An aggregation turns many values into one.

The PivotTable, VLOOKUP and filter you know, and what pandas calls each

	In Excel	In pandas	Industry term
	**PivotTable rows box**	groupby(branch_id)	The grouping key: the variable that makes the groups
	**PivotTable values box: Sum, Count, Average**	sum, count, mean	Aggregate functions (aggregation)
	**VLOOKUP or XLOOKUP**	merge	Join on a join key (a many-to-one lookup)
	**A lookup column next to your full list (IFERROR hides the #N/A)**	merge with how=left	Left join: keep every left record, NaN where no match
	**PivotTable with a field across the top**	pivot_table	Wide format (long format is the opposite)

Each pandas name is a real function name you will meet in documentation and in job tests.

Named aggregation gives several answers per group in one step

```
# orders.csv is already loaded as orders
summary = orders.groupby('branch_id').agg(
    orders=('order_id', 'count'),
    revenue=('order_value_ugx', 'sum'),
    average=('order_value_ugx', 'mean'))
print(summary.round(0).astype(int).head(4))
```

```
           orders    revenue  average
branch_id
B01          5885  283331305    48145
B02          4830  238222000    49321
B03          3345  157490190    47082
B04          3562  176738325    49618
```

In plain words

groupby names the variable that makes the groups: one group per branch.

Each line inside agg is a new column: its name, then the variable and the aggregate function.

Branch B01 has 5,885 orders and revenue of UGX 283,331,305.

These totals describe the 5 percent sample of till orders in the file.

count skips missing values and size counts every record

```
# orders is already loaded
check = orders.groupby('branch_id').agg(
    records=('customer_id', 'size'),
    filled=('customer_id', 'count'))
print(check.head(3))
```

```
           records  filled
branch_id
B01           5885    4071
B02           4830    3370
B03           3345    2336
```

In plain words

size counts the records in each group, filled or not.

count counts only the filled values of one variable. Walk-in orders have a missing customer_id, so filled is smaller.

In B01: 5,885 records but 4,071 filled customer_id values.

Say which you mean before you report a number of orders.

VLOOKUP returns the first match and merge returns every match

VLOOKUP in Sheets

Finds the first match and stops.

Fills one helper column at a time.

A missing key shows #N/A.

merge in pandas

Returns every match, one row per pair.

Adds all the variables you pick in one line.

A missing key shows NaN.

Returning every match is useful, and it means the row count can change.

A join type decides which records without a match survive

Left table: orders

order_id	customer_id
O000001	C0495
O000002	NaN
O000004	C1815

Right table: customers

customer_id	loyalty_tier
C0001	Basic
C0495	Basic
C1815	Basic

Join key: customer_id

inner join: how='inner' — 2 rows

order_id	customer_id	loyalty_tier
O000001	C0495	Basic
O000004	C1815	Basic

Only keys found in both tables.

left join: how='left' — 3 rows

order_id	customer_id	loyalty_tier
O000001	C0495	Basic
O000002	NaN	NaN
O000004	C1815	Basic

Every left record. No match gives NaN.

right join: how='right' — 3 rows

order_id	customer_id	loyalty_tier
NaN	C0001	Basic
O000001	C0495	Basic
O000004	C1815	Basic

Every right record. No match gives NaN.

full outer join: how='outer' — 4 rows

order_id	customer_id	loyalty_tier
NaN	C0001	Basic
O000001	C0495	Basic
O000004	C1815	Basic
O000002	NaN	NaN

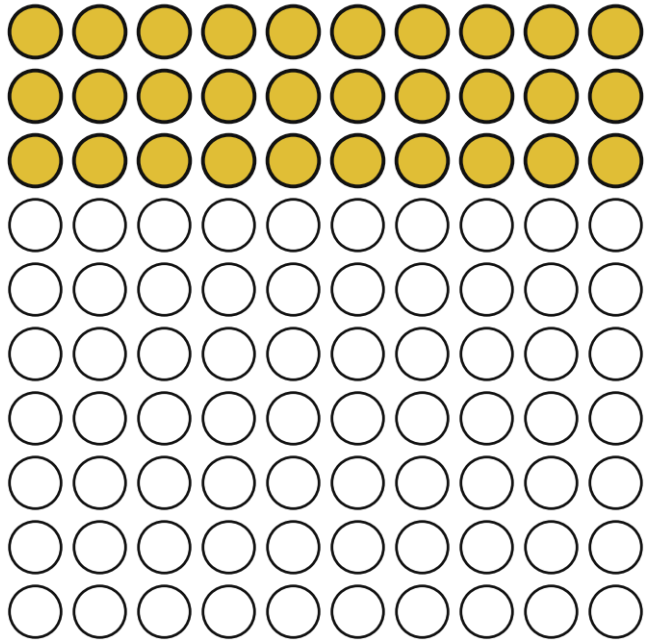
Every record from both tables.

A real excerpt, joined on customer_id. Gold cells are NaN (missing).

An inner join to customers loses 9,171 orders, the walk-in sales

30%

of the 30,229 orders have no customer_id. An inner join with customers keeps 21,058 rows. The 9,171 lost rows hold UGX 436,795,845 of revenue.



orders in every 100 are lost by the inner join

Cardinality tells you in advance whether a merge keeps, loses or multiplies rows

	Cardinality	Lakeside example	What happens to the row count
	**many-to-one (m:1)**	orders to branches on branch_id: each branch appears once.	Safe. Every order keeps one row: 30,229 stays 30,229.
	**one-to-many (1:m)**	orders to order_lines on order_id: about 3.2 lines per order.	Fan-out. 30,229 orders become 98,066 rows.
	**many-to-many (m:m)**	order_lines to returns on order_id: both sides repeat the key.	Rows multiply: the lines of a returned order meet each of its returns. An inner join gives 10,801 rows, about 5 times the 2,056 returns. Join on order_id and product_id.

Check first: is the join key unique on each side? A key that is unique on one side only is one-to-many.

Fan-out repeats the order value on every line, so a sum counts it several times

orders: one record

order_id	order_value_ugx
O000006	44,900

one-to-many

order_lines: 3 records for this order

line_id	order_id	line_revenue_ugx
L000015	O000006	30,100
L000016	O000006	7,000
L000017	O000006	7,800

```
orders.merge(order_lines, on='order_id')
```

Result: 3 rows. The order value is repeated on every row.

order_id	order_value_ugx	line_id	line_revenue_ugx
O000006	44,900	L000015	30,100
O000006	44,900	L000016	7,000
O000006	44,900	L000017	7,800

Sum of order_value_ugx after the merge: 134,700. True order value: 44,900.

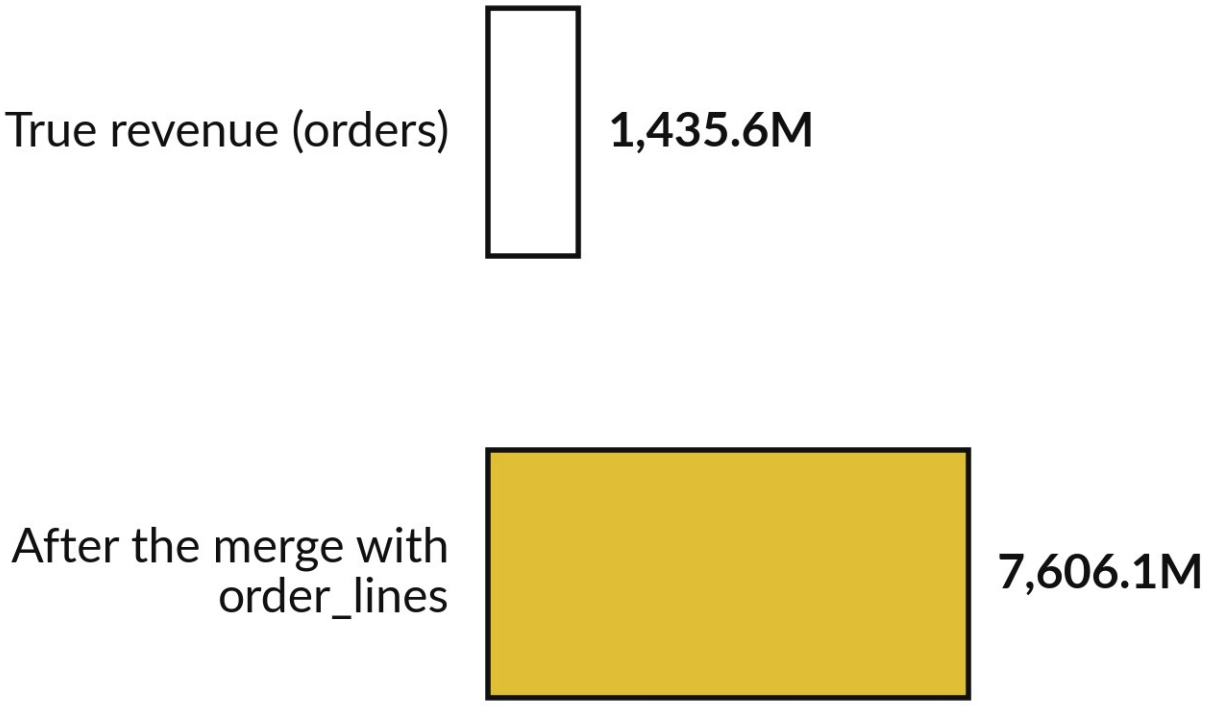
Fan-out (row multiplication): a one-to-many merge repeats the left record once for every match.

Order O000006 has 3 lines, so it becomes 3 rows.

order_value_ugx belongs to the whole order, so adding it up now counts it once per line.

No error message appears.

On the full file, fan-out makes revenue 5.3 times too big



30,229 orders became 98,066 rows after the merge.

True revenue: UGX 1,435,614,185.

Inflated total: UGX 7,606,090,055.

Ratio 5.3, above the 3.2 times rows grew: big orders have more lines.

validate='m:1' makes pandas stop a merge that is not many-to-one

```
# orders and lines are already loaded
try:
    orders.merge(lines, on='order_id',
                 validate='m:1')
except pd.errors.MergeError as err:
    print(*str(err).split('; '), sep='\n')
```

```
Merge keys are not unique in right dataset
not a many-to-one merge
```

In plain words

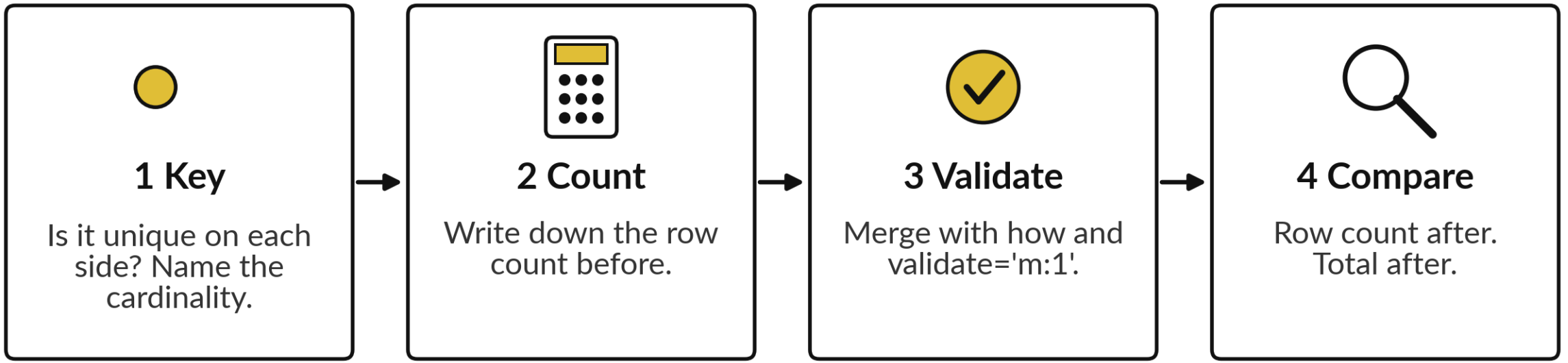
validate states the cardinality you expect: m:1 is many-to-one.

order_lines repeats order_id, so the right side is not unique and pandas raises a MergeError.

The merge never runs, so the revenue cannot inflate.

The other values are '1:1', '1:m' and 'm:m'.

Four checks around every merge protect the total



Join log (an audit trail for merges): a small table with the step, rows before, rows after and a note, like the data cleaning log. A change you cannot explain is a stop sign.

A safe lookup: many-to-one, left join, row counts before and after

```
# orders and branches are loaded
before = len(orders)
named = orders.merge(branches, on='branch_id',
                     how='left', validate='m:1')
print(before, len(named), len(named) == before)
```

```
30229 30229 True
```

In plain words

how=left keeps every order. validate says the join key must be unique in branches.

The printed line shows the count before, the count after and whether they are equal.

30,229 orders in, 30,229 rows out: nothing lost, nothing multiplied.

Add a line to your join log every time.

To use the many side, summarise it first and then merge one-to-one

```
# orders and lines are loaded
per_order = lines.groupby('order_id').agg(
    n_lines=('line_id', 'count'),
    lines_total=('line_revenue_ugx', 'sum')
).reset_index()
checked = orders.merge(per_order, on='order_id',
                       how='left', validate='1:1')
a = checked.lines_total.sum()
b = checked.order_value_ugx.sum()
print(len(checked), a == b)
```

```
30229 True
```

In plain words

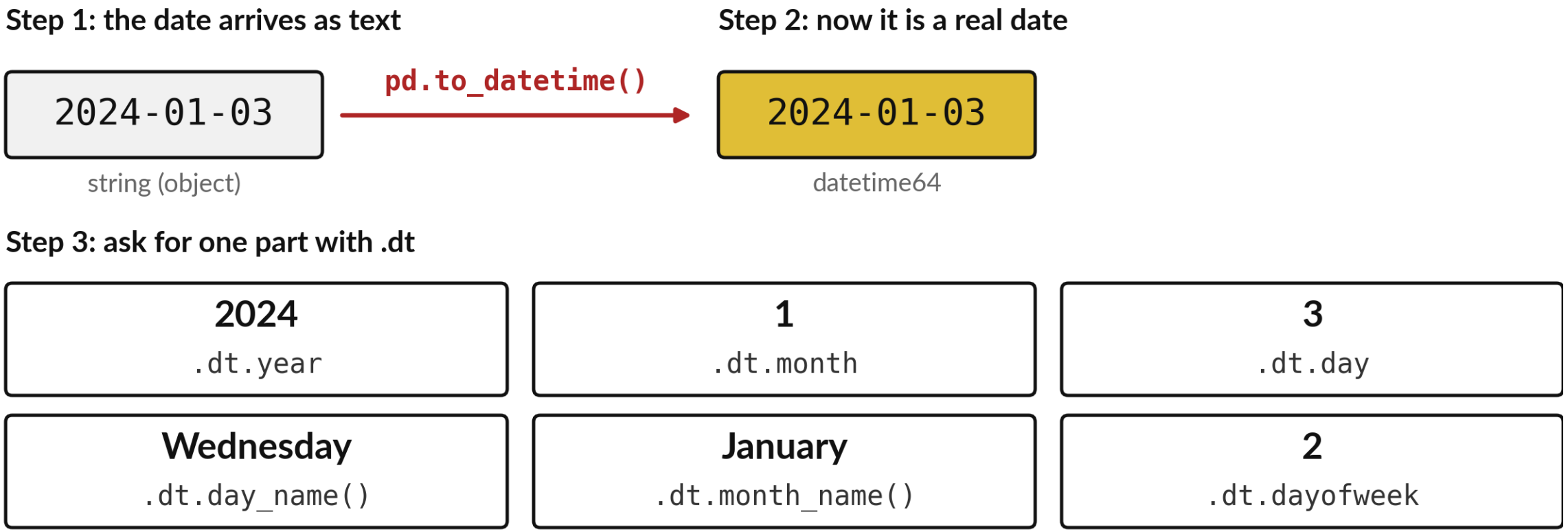
groupby reduces the lines to one record per order, with the number of lines and their total.

validate='1:1' confirms the key is unique on both sides.

30,229 rows come out. The total of the lines equals the total of the orders: True. The == test is safe here because both totals are whole shillings; with decimals use abs(a - b) < 0.01.

The pattern: summarise the many side, then merge.

Convert text to a datetime first, and then read its parts with .dt



Order O000006 was placed on 2024-01-03. dayofweek counts Monday as 0.

Revenue by weekday: derive the part, group by it, then put it in calendar order

```
import calendar
d = pd.to_datetime(orders['order_date'])
orders['weekday'] = d.dt.day_name()
by_day = (orders.groupby('weekday')
          .order_value_ugx.sum())
by_day = by_day.reindex(list(calendar.day_name))
print(by_day.idxmax(), by_day.max())
```

```
Thursday 223743120
```

In plain words

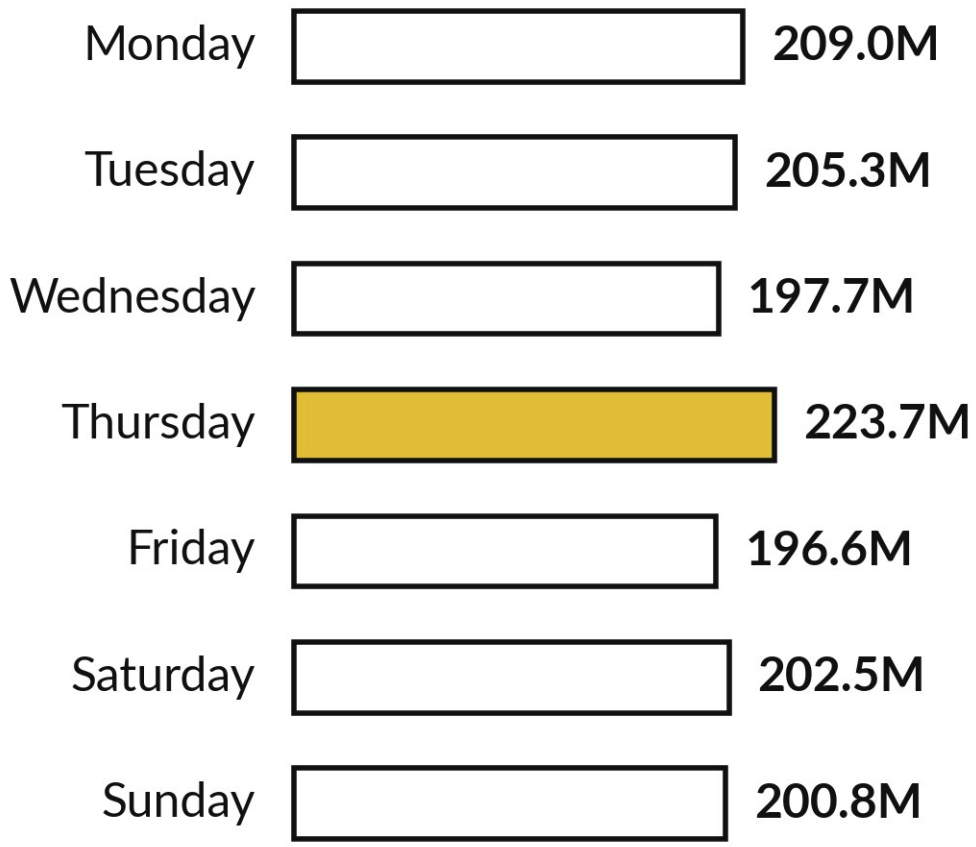
orders is already loaded. dt.day_name() gives the weekday name of each order.

groupby adds up the order value for each weekday.

reindex puts the rows in calendar order, Monday first. Without it they sort alphabetically.

idxmax gives the weekday with the largest total. max gives the total.

Thursday has the highest weekday revenue, about 1.14 times the lowest



Revenue in millions of UGX, by weekday, for the whole orders file.

Highest: Thursday, UGX 223,743,120.

Lowest: Friday, UGX 196,550,520.

This describes the sample. We have no test yet to say if the gap is more than chance.

pivot_table lays a summary out as a grid, in wide format

```
# named: the merged table from the safe lookup
d = pd.to_datetime(named['order_date'])
named['year'] = d.dt.year
table = named.pivot_table(
    index='region', columns='year', margins=True,
    values='order_value_ugx', aggfunc='sum')
print(table.astype(int))
```

```
year            2024       2025         All
region
Central    334294220  344749275   679043495
Eastern    166903605  152292430   319196035
Northern    78026570   62763125   140789695
Western    150653100  145931860   296584960
All        729877495  705736690  1435614185
```

In plain words

index: the variable down the side. columns: the variable across the top.

values and aggfunc: what to add up. margins=True adds the totals, labelled All.

Central region in 2024: UGX 334,294,220.

All branches: UGX 729,877,495 in 2024 and UGX 705,736,690 in 2025.

pivot_table makes wide format and melt makes long format

Long format: one record per branch and year

branch_id	year	revenue_ugx
B01	2024	133,419,665
B01	2025	149,911,640
B02	2024	119,475,215
B02	2025	118,746,785
B03	2024	81,399,340
B03	2025	76,090,850

`pivot_table` →

← `melt`

Wide format: one record per branch

branch_id	2024	2025
B01	133,419,665	149,911,640
B02	119,475,215	118,746,785
B03	81,399,340	76,090,850

The values are the same. The year moves from a row value to a column heading.

Long format suits groupby and charts. Wide format suits reading by eye.

Revenue of three branches by year. The numbers are the same in both layouts.

Where you will use this at work: totals you can defend

Your job	The tables you meet	What today lets you do
Credit analyst at a bank in Kampala, using pandas	loans: one record per loan (8,000 here). A real bank also has a borrowers table.	Merge with validate many-to-one, so no loan is counted twice, and report by branch and month.
Sales analyst at a retail chain, handing over to Power BI	Orders, lines, products, branches.	Build one clean table of revenue by branch, category and month, with its join log.
Monitoring officer at a district office or NGO	Clinic reports: one record per district per month (720 in our file).	Pivot to wide format for the quarterly report.
Finance officer at a SACCO or coffee cooperative	Member register and deliveries.	Left join to list members with no deliveries, and tell lost rows from zeros.

Read the recipe before the lab: five lines do most of the work

```
orders.groupby('branch_id')['order_value_ugx'].sum()
named = orders.merge(branches, on='branch_id',
                     how='left', validate='m:1')
len(orders) == len(named)
d = pd.to_datetime(named['order_date'])
named.pivot_table(
    index='region', columns=d.dt.year,
    values='order_value_ugx', aggfunc='sum')
```

In plain words

First: one revenue total for each branch.

Second: add the branch details. validate stops a merge that would multiply rows.

Third: the row count check. It must say True.

Fourth: make the date a real datetime.

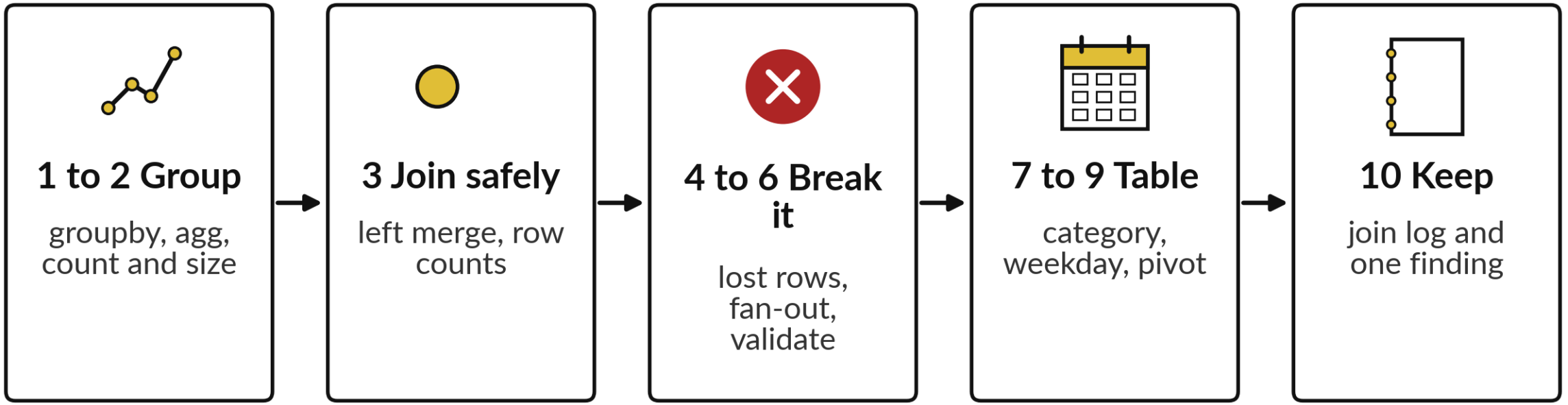
Fifth: a grid of region by year.



Part 2: the Colab lab

Close the slides. Open the notebook.

The lab has ten steps and ends with a join log you keep



Ten short tasks in 45 minutes. In each one, predict, run, read the output, and change one thing.

Check your numbers before you move on

	What you compute	You should get
	**Revenue of branch B01, and of all eight branches**	283,331,305 and 1,435,614,185
	**Orders and filled customer_id in B01 (size, count)**	5,885 and 4,071
	**Rows after the left merge to branches**	30,229: the same as before
	**Orders lost by an inner join to customers**	9,171, which is 30 percent
	**Rows after merging orders to order_lines, and the inflated revenue**	98,066 rows and 7,606,090,055, which is 5.3 times too big
	**Revenue on the weekday with the highest total**	Thursday, 223,743,120
	**Region by year pivot: Central 2024 and all branches 2025**	334,294,220 and 705,736,690

If your number differs, find the task that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**A row count after the merge is bigger than before**	The right table repeats the key: fan-out.	Add validate='m:1'. Find which key repeats. Summarise the right table first.
✗	**KeyError: 'branch_name'**	The variable is not in the table you are using.	Print the column names with .columns and check the spelling.
✓	**MergeError: not a many-to-one merge**	That is validate working correctly.	Read the message. Decide whether the merge is wrong or the expectation is.
📅	**Weekday rows are in alphabetical order**	groupby sorts the keys.	Add .reindex with the list of weekday names.
✗	**Totals double after running a cell twice**	The cell merged into a table that was already merged.	Rerun from the cell that loads the files.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**Order O000006 has 3 lines and an order value of 44,900. What do you get when you add up order_value_ugx after merging orders to order_lines?**	134,700, which is 3 times too big, because the order value is repeated on each of the 3 rows.
	**You must list every order with its customer tier. Which join type, and why?**	A left join from orders to customers. It keeps every order, including walk-ins, who get NaN for the tier.
	**Which option makes pandas refuse a merge where the right table repeats the key?**	validate='m:1' (many-to-one). It raises a MergeError before the merge runs.

Save your notebook with its join log

What you keep from today

Keep your Colab notebook: the revenue tables by branch, region and year, a join log with the row count before and after every merge, and one written finding.

Our question today

How do we answer a question that needs several groups and several tables, and trust the total?

Next session

Session 10: SQL for analysts. The same grouping and joining, written in the language every database understands.

Take away

Your lab file

A join log

The habit

Count rows twice

Next session

SQL for analysts